Physics-Informed Neural Operator-Based Poisson Solver

DeepONet + Fourier Neural Operator for 2-D Electrostatic PIC Simulations

Dhruvil Patel Om Patel

ID: 202301201 ID: 202301163

Learning the Poisson operator $\rho \mapsto \phi$ for repeated electrostatic PIC field solves

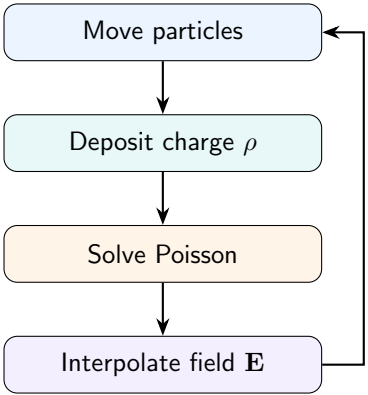
Motivation: PIC Field Solve Bottleneck

Why this matters

In an electrostatic Particle-in-Cell simulation, particle motion and electric fields are coupled. After particles move, their charge is deposited on the grid, a new potential is obtained from Poisson's equation, and the electric field is interpolated back to the particles.

Runtime bottleneck

The field solve is repeated at every time-step and can consume more than 30% of the total simulation time. Reducing this cost directly improves the feasibility of longer plasma simulations.



Governing Equation

The field solve converts a grid-based charge density into an electrostatic potential under grounded wall boundaries.

2-D electrostatic Poisson equation

$$\nabla^2 \phi(x, y) = -\frac{\rho(x, y)}{\epsilon_0}$$

Known quantities

- The PIC code supplies the charge density $\rho(x, y)$ on a 256×256 mesh.
- The physical wall condition is grounded Dirichlet: $\phi = 0$ on $\partial\Omega$.

Required output

- The solver returns the potential $\phi(x, y)$ over the same grid.
- The particle pusher then uses $\mathbf{E} = -\nabla\phi$ to update particle motion.

Why Classical Solvers Are Expensive

Classical solvers are strong reference methods, but the PIC loop uses them in the most repetitive possible setting.

Classical reference solvers

- LU/Pardiso gives reliable reference potentials, but the factorized sparse system becomes expensive as the mesh size grows.
- Iterative methods such as Conjugate Gradient reduce cost in some regimes, but still need repeated iterations and input-dependent runtime.

Why this suggests learning

- The Poisson operator and boundary condition stay fixed throughout the simulation.
- Only the input charge field ρ changes from one time-step to the next.
- This makes the problem suitable for amortized operator learning.

ρ changes
every time step

Operator fixed
same PDE and BCs

Opportunity
learn $\rho \mapsto \phi$

Why Standard Neural Surrogates Are Limited

A learned replacement must respect the global nature of electrostatics and the repeated-input setting of PIC.

CNN

- Learns a grid-to-grid map from ρ to ϕ.
- Local filters are not naturally matched to the global coupling in electrostatics.
- The learned map is tied closely to the training discretization.

Standard PINN

- Usually learns one solution surface for one source term.
- In a PIC loop, the source term changes continuously.
- Retraining or reformulating for each charge map is not practical.

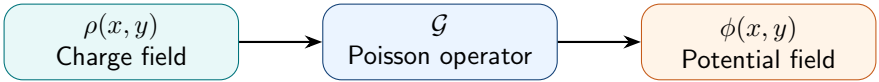
Key limitation

The repeated PIC use case needs a learned **operator**, not a single learned solution.

Reference: CNN solver paper

Operator Learning Idea

Instead of learning one potential field, the model learns the rule that maps any admissible charge field to its corresponding potential field.



Learn $\mathcal{G} : \rho \mapsto \phi$ from many charge-potential pairs.

Model choice

DeepONet provides the branch–trunk operator-learning structure, while the Fourier Neural Operator branch captures non-local electrostatic coupling efficiently in the frequency domain.

Main Contributions

1. **DeepONet Architecture:** Learns the 2D electrostatic Poisson map $\rho \mapsto \phi$, replacing repeated algebraic solves with a high-speed, fixed-cost forward pass.

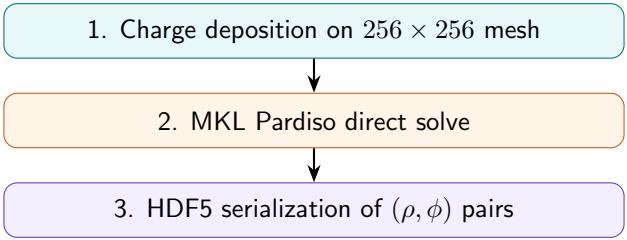
2. **Hybrid Global-Local Encoding:** Utilizes an FNO-based branch to encode global charge structure, while a coordinate trunk enables evaluation at continuous spatial coordinates.

3. **Physics-Informed Training:** Combines pointwise data accuracy with a normalized finite-difference Poisson residual and gradient alignment for superior electric-field recovery.

4. **Boundary Enforcement & Validation:** Implements hard clamping for exact grounded wall conditions, validated against 4,002 unseen plasma scenario charge fields.

Dataset Generation Pipeline

The training targets are generated from the same PIC setting in which the surrogate is later used.



- Using the C++ SYCL PIC code keeps the data distribution aligned with live simulation inputs.
- HDF5 stores the (ρ, ϕ) pairs without text conversion loss, preserving floating-point precision.

Plasma Scenarios

The dataset is built from multiple plasma regimes so that the operator sees more than one charge-field pattern.

Case	Physical setting	Training significance
1	Electric field only	Pure electrostatic electron-ion interaction
2	High magnetic field	Introduces $E \times B$ drift physics
3	Collision, low pressure	Low-pressure collisional transport
4	Collision, high pressure	High-pressure collisional transport
5	Ionisation, high pressure	Ionisation-driven validation regime
6	Collision + B field, low pressure	Test generalization with magnetic field
7	Collision + B field, high pressure	Test high-pressure magnetized regime

Split

Cases 1-4 are used for training, Case 5 for validation, and Cases 6-7 for testing.

Reference: Dataset generation paper.

Why Normalization Is Necessary

Numerical issue

- The raw charge density and potential are measured in different physical units.
- Their numerical ranges are very different, so an unscaled objective is poorly balanced.
- Standard-deviation scaling keeps localized charge spikes visible while bringing fields into trainable ranges.

Standard-deviation scaling

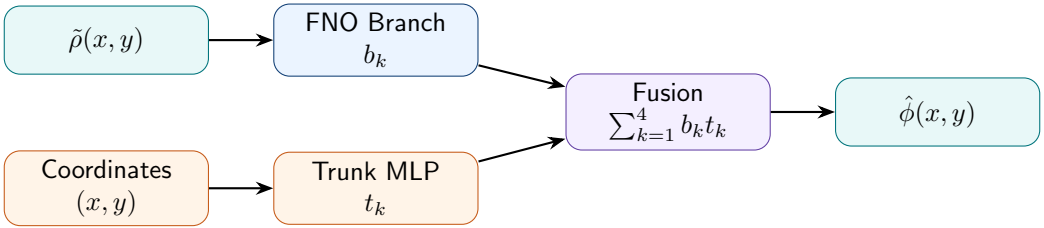
$$\tilde{\rho}_{i,j} = \frac{\rho_{i,j}}{\rho_{\text{norm}}}, \qquad \tilde{\phi}_{i,j} = \frac{\phi_{i,j}}{\phi_{\text{norm}}}$$

$$\nabla^2 \tilde{\phi} = -C\tilde{\rho}, \qquad C = \frac{\rho_{\text{norm}}}{\epsilon_0 \phi_{\text{norm}}}$$

The PDE structure is preserved; only the scale of the source term changes.

DeepONet Architecture: High-Level View

The model separates the input-dependent physics from the coordinate-dependent evaluation of the solution.



$$\hat{\tilde{\phi}}(x, y; \rho) = \sum_{k=1}^{4} b_k(\rho; x, y)\, t_k(x, y)$$

Interpretation

The FNO branch reads the normalized charge field, the trunk supplies coordinate weights, and the fusion step reconstructs the normalized potential.

Key Architecture Choices

Only the configuration choices that affect the research argument are shown here: how the model sees the charge field, how it queries space, and how it keeps the reconstruction compact.

FNO branch

- Encodes the full charge-density field in Fourier space.
- Represents non-local electrostatic coupling without relying only on local filters.

Coordinate trunk

- Receives continuous spatial coordinates.
- Supplies location-dependent weights for the branch output.

Multi-basis fusion

- Several branch fields and trunk weights are multiplied and summed.
- This gives a compact reconstruction of the normalized potential.

FNO Branch: Why Fourier Space?

Reason for using FNO

The Poisson equation is non-local: potential at one point depends on charge throughout the domain. A Fourier layer sees the full field through its frequency representation, so global coupling is built into the branch network.

One FNO spectral operation

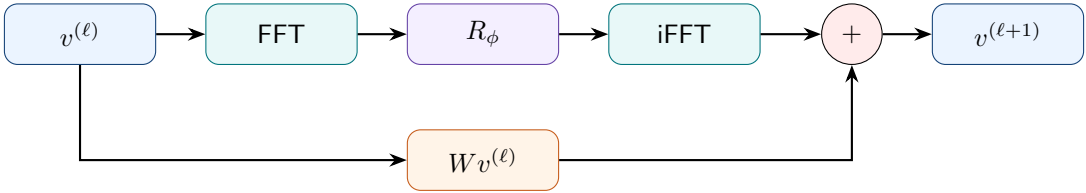
$$(\mathcal{K}u)(x) = \mathcal{F}^{-1}\left(R_\phi \cdot (\mathcal{F}u)\right)(x)$$

What is learned

- The complex weights R_ϕ mix retained spectral modes.
- A finite low-frequency mode budget focuses the model on the dominant smooth electrostatic structure while limiting memory use.

Single FNO Layer

Each FNO layer combines a global spectral path with a local pointwise path, so the branch can learn both long-range field structure and local corrections.



- The spectral path uses FFT, learned spectral weights, and inverse FFT to capture smooth long-range structure.
- The pointwise skip projection $Wv^{(\ell)}$ preserves local channel information before the two paths are added.

Trunk Network and Multi-Basis Fusion

The trunk is the part that lets the learned operator be evaluated through spatial coordinates rather than only through pixel indices.

Coordinate trunk

- Receives continuous coordinates $(x, y) \in [0, 1]^2$.
- Produces coordinate-dependent weights $t_k(x, y)$ using a compact MLP.

Multi-basis fusion

$$\hat{\tilde{\phi}} = b_1 t_1 + b_2 t_2 + b_3 t_3 + b_4 t_4$$

Each product combines charge-dependent information from the branch with geometry-dependent weights from the trunk.

The model predicts the potential ϕ first, then derives the electric field via $\mathbf{E} = -\nabla\phi$.

Hard Boundary Enforcement

Grounded walls are a physical constraint, so the final prediction should satisfy them exactly rather than only approximately.

Post-processing & Zero-Clamping

$$\hat{\phi}_{\text{corrected}} = \hat{\phi}_{\text{pred}} - \delta^{(n)}, \quad \delta^{(n)} = \text{avg}\left(\hat{\phi}_{\text{pred}}\big|_{\partial\Omega}\right)$$

$$\hat{\phi}_{\text{final}}(\mathbf{x}_{\partial\Omega}) = 0$$

Gauge correction

- Subtracting the mean predicted boundary value selects the grounded Dirichlet gauge.
- This constant shift does not change the recovered interior electric field.

PIC stability

- Hard clamping enforces exact zero-potential walls.
- This avoids residual boundary gradients that could create spurious particle acceleration.

Mesh Invariance

The intended learned object is a continuous operator, so the architecture should not behave like a simple pixel-to-pixel lookup table.

FNO branch

- Stores a fixed set of spectral weights.
- FFT and inverse FFT can be evaluated on different grid sizes.

Coordinate trunk

- Receives coordinates, not pixel labels.
- The same MLP can be queried on a finer grid.
- It learns spatial dependence through (x, y).

Interpretation

A model trained at 256×256 is therefore designed as an approximation of $\mathcal{G} : \rho \mapsto \phi$, with higher-resolution empirical testing left as future work.

Composite Loss: Data Fit Plus Physics Constraints

A data-only network may minimise potential error but still produce a field that is not Poisson-consistent. The final objective adds the missing physical checks directly into training.

Objective from the report

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{data}} + \lambda_{\text{eff}}(e)\mathcal{L}_{\text{pde}} + \lambda_{\nabla}\mathcal{L}_{\text{grad}}$$

The effective physics weight $\lambda_{\text{eff}}(e)$ is warmed up during the first five epochs, so the model first learns the coarse Pardiso-like potential before the PDE residual reaches full strength.

Data term

Matches the predicted potential with the Pardiso reference at every grid point.

$$\text{MSE}(\hat{\tilde{\phi}}, \tilde{\phi}^{\text{true}})$$

PDE term

Penalises violations of the normalized Poisson equation on interior cells.

$$r_{i,j} = -\nabla_h^2 \hat{\tilde{\phi}} - C\tilde{\rho}$$

Gradient term

Aligns spatial slopes so the derived electric field remains smooth.

$$\text{MSE}(\nabla\hat{\tilde{\phi}}, \nabla\tilde{\phi}^{\text{true}})$$

Why the PDE Residual Uses Finite Differences

The PDE residual needs second spatial derivatives. For this full-field neural operator, finite differences are more practical than automatic differentiation.

Why automatic differentiation was not used

- The model predicts the complete 256×256 potential field in one pass.
- Computing second derivatives through four FFT/iFFT layers forces PyTorch to keep a large derivative graph.
- In our experiments, this caused GPU memory failure and roughly $10\times$ slower epochs.

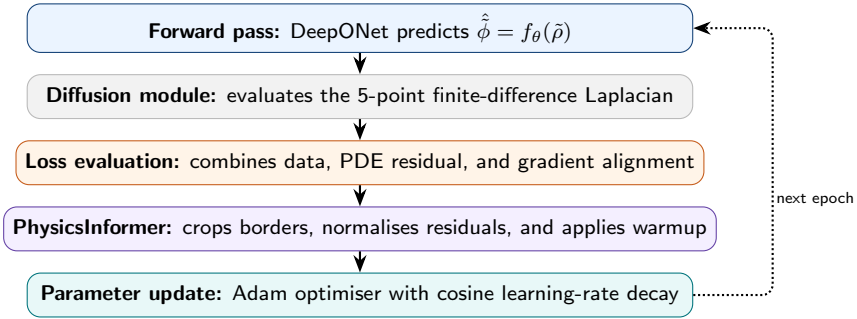
5-point Laplacian

$$\nabla_h^2 f_{i,j} = \frac{f_{i+1,j} + f_{i-1,j} + f_{i,j+1} + f_{i,j-1} - 4f_{i,j}}{h^2}$$

The residual is computed after the forward pass using detached tensor shifts. This matches the C++ PIC stencil and avoids retaining a costly automatic-differentiation graph through the spectral layers.

Training Loop: From Prediction to Parameter Update

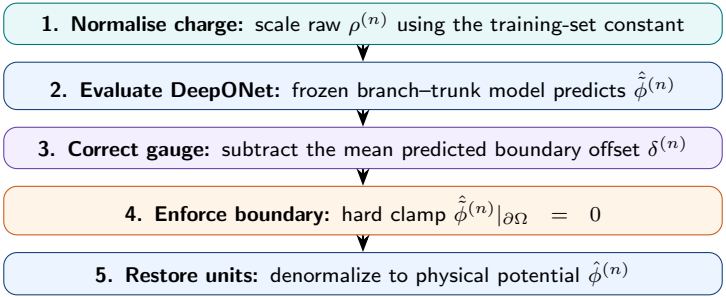
Each epoch converts one charge field into a predicted potential, evaluates data and physics errors, and updates the network with a controlled optimiser step.



Training is run from scratch, and the selected model is the checkpoint with the best validation behaviour. Gradient clipping is used to avoid unstable early updates.

Inference Pipeline: Frozen Model, Deterministic Physics Fixes

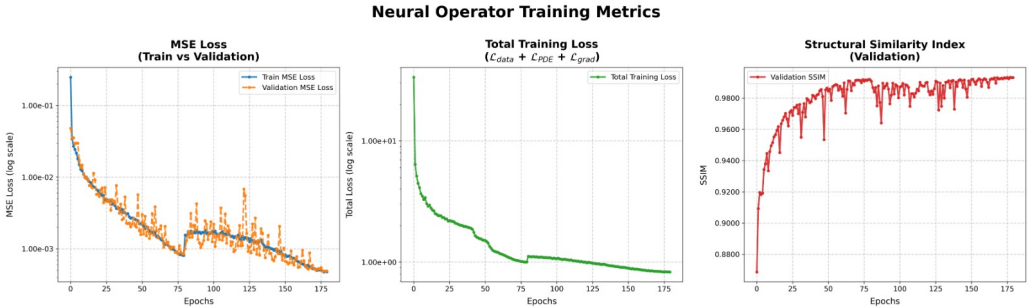
After training, no parameter updates are performed. Each new PIC charge field passes through the same deterministic sequence before the potential is used to compute the electric field.



Gauge correction removes a constant boundary offset without changing interior physics, and hard clamping guarantees $\hat{\phi} = 0$ on the grounded walls before $\mathbf{E} = -\nabla\phi$ is recovered.

Training Convergence: All Validation Signals Improve

The convergence curves show whether the model is learning only pointwise potential values or also improving structural and physics consistency over training.



The validation curves show the intended behaviour: potential error decreases, SSIM improves, APE drops, and the normalized PDE residual also reduces. The final checkpoint is selected from validation performance, not from training loss alone.

Full Testing Summary: Held-Out Charge Fields

The final checkpoint is evaluated on charge configurations that were not used for parameter updates. The summary below reports the aggregate behaviour over the full validation set.

4.2178×10^{-2} **V**
Average RMSE

3.5599%
Average APE

1.5001%
Median APE

4002
held-out fields

What the accuracy numbers show

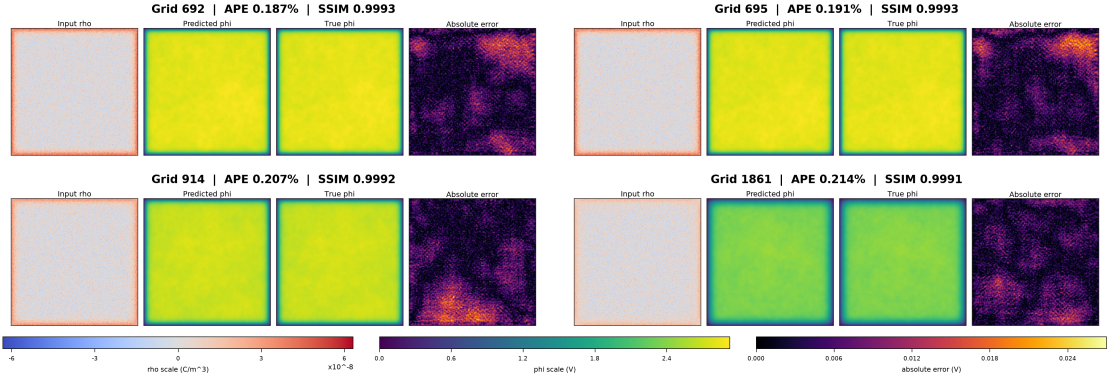
- The median APE is much lower than the mean APE.
- Most samples have small relative error.
- A smaller high-gradient tail raises the average.

How to read the residual

- The PDE residual does not reach zero because the model is not an exact algebraic solver.
- The reported value reflects the balance among data fitting, PDE consistency, and gradient alignment.

Qualitative Prediction Results

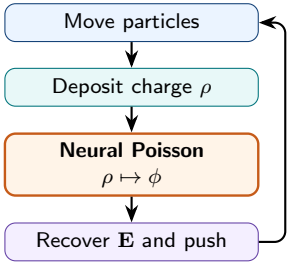
Selected validation grids show predicted potential, Pardiso reference, and absolute error using shared visible color scales.



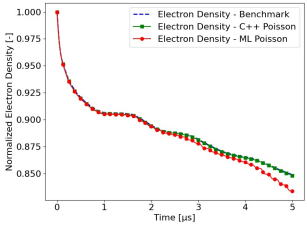
The selected samples show close agreement between $\hat{\phi}$ and the Pardiso reference, while the error maps remain localized and low in magnitude.

Dynamic Validation: Does the Solver Stay Stable in PIC?

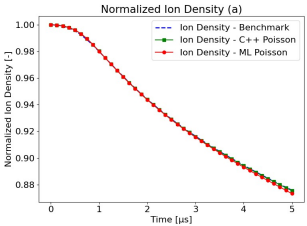
After snapshot accuracy is verified, the trained model is inserted into the live PIC loop at the field-solve step and tested as part of the coupled plasma update.



Insertion point in PIC loop



Electron density



Ion density

The neural solver replaces the classical Poisson solve inside the C++ PIC loop. Over 2000 coupled iterations, electron and ion density decay remains close to the serial C++ solver and the expected benchmark trajectory.

What the Proposed Solver Contributes

The contribution is not only a neural network replacement; it is a complete training and deployment path for a Poisson surrogate in a PIC workflow.

Operator Replacement

The model learns $\rho \mapsto \phi$ directly from PIC-generated fields, so each new charge distribution can be processed by a fixed forward pass instead of a repeated algebraic solve.

Physics Handling

Finite-difference residuals avoid the AD memory bottleneck, gradient alignment improves recovered electric fields, and hard clamping guarantees exact grounded boundaries.

Architecture Choice

The FNO branch captures non-local electrostatic coupling, while the coordinate trunk supplies spatial query weights. Four basis fields combine to reconstruct the normalized potential.

Validation

The final solver is evaluated on 4002 held-out validation fields and tested inside the PIC loop for 2000 iterations, achieving sub-4% average APE.

Conclusion

The study shows that the Poisson solve in an electrostatic PIC loop can be treated as an operator-learning problem rather than a repeated matrix solve.

4.2178×10^{-2} **V**
average RMSE

3.5599%
average APE

1.5001%
median APE

2000
PIC iterations

Method conclusion

- The branch–trunk DeepONet replaces the repeated 2-D Poisson solve with a learned $\rho \mapsto \phi$ operator.
- The FNO branch captures non-local electrostatic coupling.
- The coordinate trunk supports spatial evaluation of the learned field.

Physics conclusion

- Finite-difference physics supervision avoids the AD memory failure.
- Hard boundary clamping enforces exact grounded walls.
- This makes the surrogate compatible with the tested PIC workflow.

Main Research Impact

The central result is a practical path from a classical field-solve bottleneck to a trained, physics-constrained neural operator.

Problem

PIC simulations repeat the Poisson solve at every time-step, making the field solve a major runtime bottleneck.

Approach

DeepONet learns the continuous field operator $\rho \mapsto \phi$ using an FNO branch and coordinate trunk.

Result

The solver reaches sub-4% average APE, exact wall potential, and stable PIC-loop validation.

Contribution to remember: the solver learns the Poisson operator once, then evaluates new charge fields without retraining.

Physics-Informed Neural Operator-Based Poisson Solver

Dhruvil Patel (202301201) Om Patel (202301163)